

INFORME TÉCNICO

Firewall Dashboard

Gestión de reglas de iptables con dashboard web, sobre una VM Linux aislada

Proyecto de portfolio de ciberseguridad defensiva

Mario Pérez López

github.com/Mariio09/firewall-lab

1. Resumen ejecutivo

Firewall Dashboard es una aplicación full-stack que expone la gestión de iptables a través de un panel web: crear, aplicar y auditar reglas de firewall sobre una VM Ubuntu sin usar la línea de comandos directamente. El proyecto no pretende sustituir herramientas como ufw. Demuestra cómo construir con criterio un sistema que ejecuta comandos privilegiados a partir de entrada de usuario, con validación en profundidad, privilegios mínimos y cada decisión de seguridad documentada y explicable.

El desarrollo se organizó en tres bloques verificables de forma independiente: (A) la aplicación completa contra un firewall simulado en memoria, sin privilegios ni VM; (B) la capa de iptables real sobre la VM, incluyendo un incidente de auto-bloqueo provocado y medido a propósito; y (C) la interconexión end-to-end entre el frontend en el host y el backend en la VM, con detección de drift en ambas direcciones.

2. Qué es y por qué está construido así

El backend corre en una VM Ubuntu levantada con Multipass, aislada de la red local. El frontend, en React y TypeScript, habla con la API por HTTP con autenticación JWT. La base de datos SQLite es la fuente de verdad: una regla de firewall en este sistema no es solo una línea de iptables, es una fila con autor, fecha y motivo, información que iptables no puede representar y que importa seis meses después.

Arquitectura, a alto nivel

HOST

React + TypeScript (Vite) → HTTP + JWT →

VM: firewall-lab

FastAPI → services/ (lógica de negocio) → firewall/ (único módulo con subprocess) → SQLite + iptables (FWDASH_INPUT / FWDASH_OUTPUT / FWDASH_FORWARD)

Las dependencias van en una sola dirección: api/ → services/ → firewall/ + models/. Un router nunca importa firewall/ directamente; un servicio nunca importa fastapi; el paquete firewall/ no sabe que existe una base de datos. Dos reglas de arquitectura que un test automático verifica en cada build.

3. Decisiones de diseño (ADR)

Las decisiones con impacto arquitectónico se documentan como Architecture Decision Records, con las opciones descartadas y el porqué. El proyecto tiene 18 en total; estos son los que más pesan en el diseño y en la capa de seguridad:

ADR-0001 — La base de datos como fuente de verdad

iptables refleja lo que dice SQLite, nunca al revés. Permite auditoría, historial y metadatos que iptables no puede almacenar.

ADR-0002 — Cadenas propias y reconstrucción idempotente

La app crea FWDASH_INPUT, FWDASH_OUTPUT y FWDASH_FORWARD con un único salto desde las cadenas estándar. 'Aplicar' siempre vacía y reconstruye la cadena gestionada entera, lo que garantiza que nunca se toquen reglas ajenas al sistema.

ADR-0003 — Capabilities en lugar de sudo

NOPASSWD sobre iptables equivale, en la práctica, a acceso de root. CAP_NET_ADMIN vía systemd (AmbientCapabilities) concede exactamente el permiso necesario y nada más.

ADR-0004 — Backend de firewall intercambiable

Una interfaz (Protocol) con dos implementaciones, fake e iptables. El MVP completo (API, auth, frontend) se construye y valida en el host sin VM ni privilegios, antes de tocar una regla real.

ADR-0006 — Las reglas se comparan por estructura, nunca por texto

iptables reescribe cada regla al guardarla (--dport 22 vuelve como -p tcp -m tcp --dport 22). Comparar texto daría drift permanente incluso recién aplicado; se comparan objetos RuleSpec normalizados.

ADR-0007 — Lo que el sistema dice y no se entiende, no se descarta

Una regla dentro de una cadena gestionada que el parser no sabe expresar es exactamente el drift que hay que detectar: se marca, nunca se ignora.

ADR-0011 — Escribir la política y aplicarla son dos contratos distintos

Si crear una regla funciona pero iptables falla al aplicar, POST /rules devuelve 201 con sync_state=failed (la regla existe, la BD es la fuente de verdad); POST /firewall/apply devuelve 502, porque ahí lo único pedido era aplicar.

ADR-0015 — La aplicación arranca aunque el firewall no responda

Si iptables falla al montarse durante el arranque, la aplicación no muere: arranca degradada, el fallo queda auditado y se repite, tipado, en la primera petición a /firewall/*. Morir en el arranque dejaría inservible justo la herramienta que hace falta para diagnosticar por qué el firewall ha fallado.

ADR-0016 — El CIDR de gestión se declara, no se supone

Un valor por defecto plausible pero falso (el rango habitual de Multipass, no el real de esta máquina) hacía que el guardián de gestión abriera el puerto de la API a una subred donde no había nadie: la regla escrita para evitar el auto-bloqueo era la que lo provocaba. Ahora es obligatorio declararlo en cuanto las reglas llegan a iptables de verdad; sin él, la aplicación no arranca.

ADR-0017 — El canal de rescate se protege si se declara

Medido en B4: un DROP trivial sobre el puerto 22 corta el único canal de rescate (SSH, por la misma red que filtra el firewall) sin que la API se inmute. El dashboard sigue viéndose perfecto en el navegador mientras el operador se queda fuera de la máquina. MANAGEMENT_SSH_PORT, opcional y sin valor por defecto, añade un cuarto guardián declarado en el .env y visible en el preview, en vez de un agujero fijo e invisible en el firewall.

ADR-0018 — La política se reconcilia al arrancar

Un reinicio deja las cadenas FWDASH_* vacías aunque la base de datos conserve la política guardada: montar no es poblar. El lifespan reaplica siempre las tres cadenas al arrancar (salvo AUTO_APPLY=false), porque incluso con la política vacía lo que se escribe son los guardianes; saltarse el apply dejaría el puerto de gestión sin proteger hasta la primera aplicación manual.

4. Seguridad: contención contra inyección de comandos

Es la amenaza principal de cualquier sistema que traduce entrada de un formulario web en comandos de sistema privilegiados. Se mitiga con cinco capas independientes:

- subprocess.run(argv, shell=False), argumentos siempre en lista: sin shell, la inyección deja de ser posible por construcción, no por disciplina.
- subprocess se importa en un único archivo de todo el repositorio.
- Allowlist explícita de binarios ejecutables (ningún PATH implícito; los binarios auxiliares se resuelven como hermanos de iptables).
- RuleSpec inmutable, validada en su constructor: no existe ruta de código capaz de fabricar una regla inválida.
- Normalización canónica de IPs (a CIDR) y puertos antes de que lleguen a la red.

Las capas 2 y 3 no dependen de la disciplina de quien programe: un test de arquitectura en la suite de CI falla si alguien las cruza, por ejemplo si subprocess se importa desde un segundo archivo.

Validación de entrada

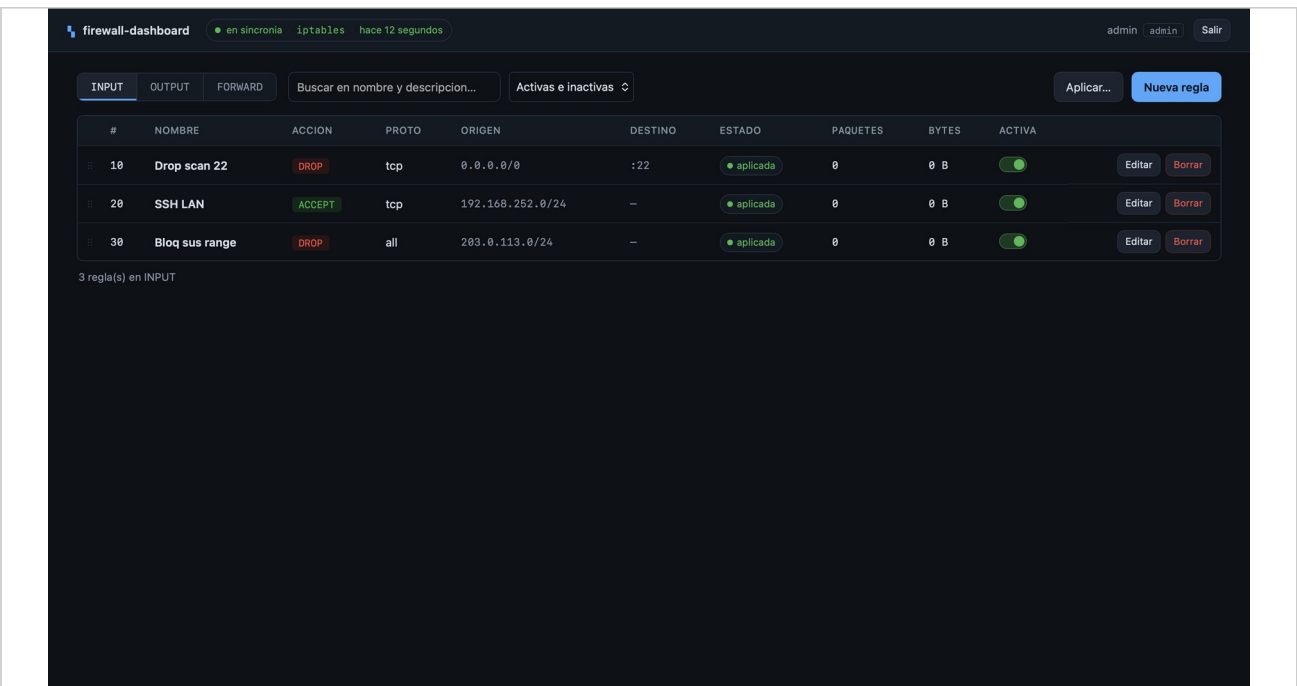
Todo dato pasa por un validador dedicado antes de tocar la red: direcciones IP y CIDR con el módulo estándar ipaddress, puertos y rangos de puerto verificados dentro del rango de 1 a 65535, nombres de interfaz por expresión regular (sin comodines como eth+), y los campos de texto libre (comentarios, prefijos de log) saneados frente a los caracteres que sí importan cuando no hay shell de por medio: comillas y caracteres de control, no `;` ni `\$`, que sin intérprete de shell no representan ningún riesgo real.

5. Autenticación y control de acceso

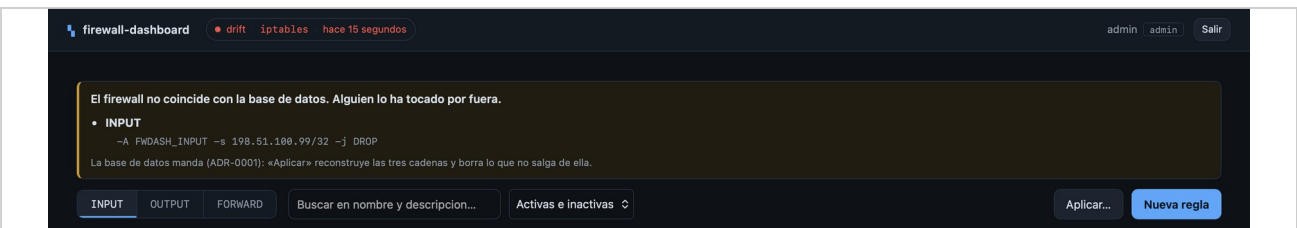
- Contraseñas con Argon2id; sesiones con JWT (PyJWT), refresh token sin estado en servidor.
- RBAC en tres roles jerárquicos: admin $\supset$ operator $\supset$ viewer, verificados en la propia firma de cada endpoint.
- La identidad del usuario se relee de la base de datos en cada petición, nunca se confía solo en el contenido del token.
- Sesión del frontend en memoria, no en localStorage: recargar la página exige volver a iniciar sesión, de forma deliberada.

6. Capturas del sistema

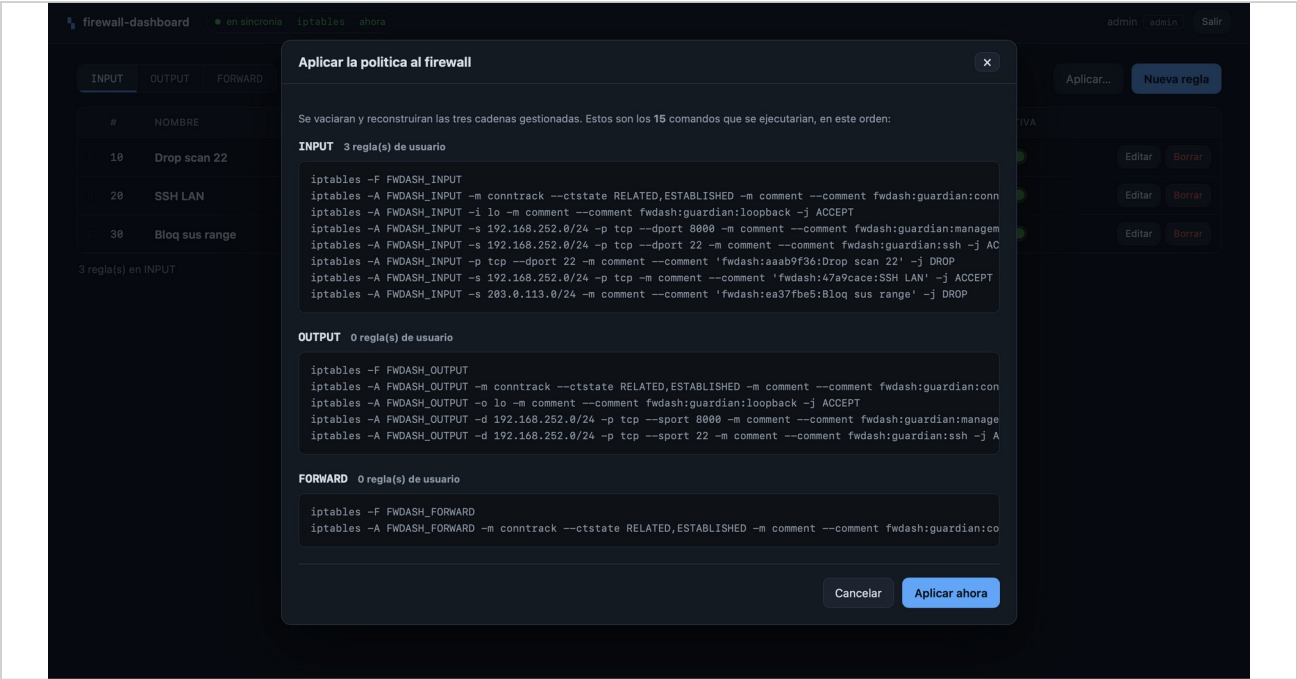
Las cuatro capturas siguientes recorren las tres capas de la aplicación de punta a punta: navegador, API y kernel de la VM.



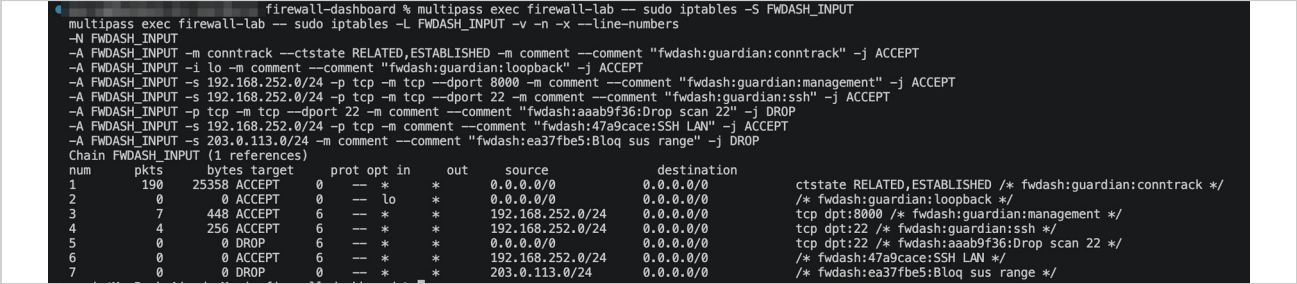
1. Dashboard. Tabla de reglas con sync_state=applied y estado del firewall en iptables



2. Drift. Banner de drift activo tras un cambio manual en la VM



3. Preview de aplicación. Comandos iptables exactos antes de confirmar, guardianes incluidos



4. Verificación cruzada. La misma regla vista en iptables -S dentro de la VM

7. Stack tecnológico

Backend: FastAPI · SQLAlchemy 2.0 · Alembic · SQLite · structlog · Argon2 · PyJWT · pytest · ruff · mypy · bandit

Frontend: React 18 · TypeScript · Vite · TanStack Query

Infraestructura: VM Ubuntu vía Multipass · systemd (CAP_NET_ADMIN) · iptables

8. Estado del proyecto

Bloque	Contenido	Estado
A	Aplicación completa contra firewall en memoria	Cerrado: núcleo, capa firewall/, auth, API de reglas y frontend
B	Capa de iptables real, privilegios, runbook	Cerrado: runner con allowlist, IptablesBackend, capabilities, auto-bloqueo provocado y medido, suite de contrato contra iptables real
C	Interconexión, drift real, recorrido completo	En curso: código y arnés de verificación listos; cierra con las capturas y el tag v0.1.0-mvp

Bloque	Contenido	Estado
Fase 2	Logging de paquetes bloqueados, rollback con confirmación	Planificado
Fase 3	Dashboard con gráficas por IP / puerto / tiempo	Planificado
Fase 4	Detección de patrones y alertas	Planificado

Repositorio: github.com/Mariio09/firewall-lab